01-2-README

accounts-api security assessment · generated from the repository, not hand-edited

> **Section 1 deliverable: README covering how a reviewer runs the policy tests.**
>
> Assembled by `scripts/build-submission.py` from the repository. Source paths are given above each section.

`README.md`

accounts-api — security engineering submission

Securing a containerised account-data service on shared EKS in `ap-south-1`: the pipeline that builds it, the identities it assumes, the secrets it holds, and the controls a bank is examined against.

Two things a reviewer can check in minutes rather than take on trust:

- **The policy suite runs offline.** No AWS account, no credentials, no `terraform init`, no registry, no cluster. One command, eleven checks, three of which pass *by failing*.
- **The supply-chain pipeline actually runs.** Push this repo and the Actions tab shows a real green run that builds the image, generates the SBOM, keyless-signs it, attests it, and then **verifies its own signature** — against GHCR, with no AWS account and no secrets. The AWS half is written as it would ship and is gated behind one repository variable.

Nothing was applied against a real cloud. Where production would use AWS, the mapping is documented and the local equivalent is what runs. `ASSUMPTIONS.md` lists every gap I filled and exactly what was and was not executed.

Run the policy tests

```
# 1. tools (four binaries, ~60 MB, no package manager needed)
mkdir -p bin && cd bin
curl -fsSL https://github.com/kyverno/kyverno/releases/download/v1.13.4/kyverno-
cli_v1.13.4_linux_x86_64.tar.gz | tar xz kyverno
curl -fsSL https://github.com/open-policy-
agent/conftest/releases/download/v0.56.0/conftest_0.56.0_Linux_x86_64.tar.gz | tar xz conftest
curl -fsSL -o cosign https://github.com/sigstore/cosign/releases/download/v2.4.1/cosign-linux-amd64
&& chmod +x cosign
cd .. && export PATH="$PWD/bin:$PATH"
pip install pyyaml

# 2. everything
./scripts/run-policy-tests.sh
```

Expected output — eleven checks, all passing, three of which pass **by failing**:

```
──── fixture drift
  good-workload fixture == remediate/workload-fixed.yaml    PASS
──── Kyverno admission policies  (section 4a + 5)
```

```
  kyverno test — 3 fixtures, 39 assertions                  PASS
─── Terraform policy-as-code  (section 4b)
  conftest verify — 33 rego unit tests                      PASS
  bad fixture must be rejected (11 deny rules)              PASS (failed as intended)
  evasion fixture must be rejected (var-driven, dynamic, IPv6, Aurora)  PASS (failed as intended)
  good fixture must pass                                    PASS
  shipped terraform/ must pass                              PASS
─── Waiver mechanism  (section 3)
  valid waivers: schema, approver, lifetime, expiry, signature  PASS
  expired / self-approved / over-cap / unwaivable-secret    PASS (failed as intended)
  post-approval edit of `expires` breaks the signature      PASS (failed as intended)
  derived ignore files come only from ACTIVE waivers        PASS
```

Individual commands, if you would rather run them yourself:

```
kyverno test policy/kyverno/tests                                              # 39 assertions
conftest verify --policy policy/rego                                           # 33 rego unit tests
conftest test --policy policy/rego --parser hcl2 --all-namespaces terraform/   # must pass
conftest test --policy policy/rego --parser hcl2 policy/fixtures/terraform/bad      # must FAIL, 11
denies
conftest test --policy policy/rego --parser hcl2 policy/fixtures/terraform/evasion  # must FAIL, 7
denies
python3 scripts/waiver_check.py check --verify-signatures                      # must pass
python3 scripts/waiver_check.py check --file waivers/fixtures/tampered.yaml --verify-signatures  #
must FAIL
```

Captured output from a real run is committed in `test-output/`.

> **This suite is designed to go red on 2026-09-27.** Waiver `SEC-2041` expires then, and the whole point of the mechanism is that the build fails when a waiver lapses rather than quietly continuing to suppress the finding. If you are reading this after that date, that failure is the control working — the real fix is a renewal PR. To see the rest of the suite green regardless: `WAIVER_TODAY=2026-09-04 ./scripts/run-policy-tests.sh`

Optional: against a real cluster

```
./scripts/run-policy-tests.sh --integration   # prints the kind + Kyverno recipe
```

Needed for exactly one thing: `verify-image-signature`, which cannot be unit-tested because the CLI cannot verify a Sigstore signature without registry and Rekor access. It has **no assertion** in the offline suite rather than a fabricated passing one.

Where things are

Read in order. Start with the threat model; everything else is downstream of it.

`THREAT-MODEL.md`	one page: trust boundaries, ranked top-5 threats T1–T5, controls or acceptance for each, and the threat I think is overrated here
`TRACEABILITY.md`	every artefact in the repo mapped back to a threat — including the two built without one, and why

`DECISIONS.md`	**Part 2.** biggest decisions, enforcement posture, two blast-radius scenarios, the attack path I cannot detect, regulatory table, what I cut
`ASSUMPTIONS.md`	every gap in the brief I filled, and a plain statement of what was executed and what was not
`docs/07-operating-model.md`	who owns each control, the four numbers I would put on a dashboard, review cadence, and the failure modes that kill a control in month twelve

section	build artefacts	write-up
2 — build & release	`.github/workflows/build-release.yml`, `iam/github-oidc-trust-policy.json`, `terraform/ecr.tf`	`docs/02-build-integrity.md` — incl. **what the signature does not prove**
3 — gates & waivers	`.github/workflows/security-gates.yml`, `waivers/`, `scripts/waiver_check.py`	`docs/03-gates-and-waivers.md`, `waivers/README.md`, `docs/03-break-glass.md`
4 — review & remediate	`remediate/workload-fixed.yaml`, `terraform/`, `iam/pod-identity-policy.json`, `policy/kyverno/`, `policy/rego/`	`docs/04-findings.md` — defects ranked by exploitability, not scanner severity
5 — admission & runtime	`policy/kyverno/policies/`, `runtime/falco-rules.yaml`, `k8s/networkpolicy-egress.yaml`, `k8s/istio-egress.yaml`	`docs/05-admission-rollout.md`, `docs/05-egress.md`, `runtime/runbook-accounts-api.md`
6 — secrets & data	`k8s/externalsecret.yaml`, `terraform/kms.tf`, `observability/fluent-bit-redaction.conf`	`docs/06-secrets-rotation.md`, `docs/06-log-redaction.md`

The three broken artefacts from the brief are committed verbatim as negative fixtures — `policy/kyverno/tests/resources/bad-workload.yaml`, `policy/fixtures/terraform/bad/main.tf`, `iam/pod-identity-policy-original.json` — so the "fails before the fix, passes after" claim is executable rather than asserted.

Three things worth a reviewer's attention

The waiver mechanism is the part I built most carefully, because a gate with no waiver path gets disabled within a month. Waivers are in-repo, **cosign-signed over a canonical body** (so `expires` cannot be quietly extended after approval — `waivers/fixtures/tampered.yaml` proves it), capped by severity (CRITICAL 14d → LOW 180d), two-person with an approver authority matrix, and **fail-closed on lapse**: `.trivyignore`, `.gitleaks.toml` and friends are `.gitignore`d and *generated* from unexpired waivers at pipeline time, so an expired waiver both fails the build and stops suppressing. A real secret cannot be waived at all — only rotated.

It is not theoretical: gitleaks flags one real false positive in this repository (the External Secrets CRDs), and `test-output/gitleaks-waiver-worked-example.txt` is the captured end-to-end run — finding → signed waiver → derived allowlist → clean scan — with the commands to reproduce it.

The Kyverno suite has three fixtures, not two. A broken manifest and a fixed one only prove the rules distinguish those two files. `bypass-workload.yaml` is what the manifest looks like after somebody has read the policy names and tried to satisfy them without changing the behaviour — digest-pinned but to Docker Hub,

`runAsNonRoot: true` next to `runAsUser: 0`, `seccompProfile: Unconfined`, `secretKeyRef` instead of `envFrom`, `hostPath` renamed and pointed somewhere innocuous. It must still be rejected, and the suite asserts which rule catches each.

The rules were tested against evasion, not just against the broken files. `policy/fixtures/terraform/evasion/` is the fixture that separates a rule set from a speed bump: `publicly_accessible = var.public` parses as a *string*, so a rule testing `== true` passes it; `cidr_blocks = var.allowed` parses as a string rather than a list, so a rule that iterates the list never runs; `dynamic "ingress"` hides the rules entirely; `::/0` is IPv6 for `0.0.0.0/0`; and `aws_rds_cluster` is the same database under a different type. The rule set **fails closed on anything it cannot statically prove safe**, and there is a unit test asserting that a literal `false` is still accepted — otherwise "deny the unprovable" quietly becomes "deny everything". Same story on the Kubernetes side: `initContainers` and `RoleBinding → cluster-admin` are the two blind spots a Role-only policy leaves, and both are now assertions.

The honest gaps are written down, not omitted. What the signature does not prove (`docs/02`), what the egress control still fails to stop (`docs/05-egress.md` — seven items), what the log redaction still leaks (`docs/06` — seven items), and the one attack path I would currently not detect at all (`DECISIONS.md` §4, with costed options to close it).

For the assessment form

`submission/SUBMISSION-GUIDE.md` maps every field on the assessment form to the exact file that answers it. The deliverables themselves are in `submission/files/` (native formats) and `submission/pdf/` (same content, typeset, for fields whose uploader only accepts PDF). Both are generated from this repository by `scripts/build-submission.py`, so they cannot drift from what is committed. `submission/PUSH-TO-GITHUB.md` covers publishing and what a green pipeline should look like.

AWS → local mapping

Nothing here needs a paid account. Where production is AWS:

production	local / free equivalent	what changes
ECR, `image_tag_mutability = IMMUTABLE`	GHCR, or `registry:2`	GHCR has no true tag-immutability setting. A genuine downgrade, named in `DECISIONS.md`.
IAM OIDC role for CI	`GITHUB_TOKEN` scoped `packages: write`	still short-lived and job-scoped; loses the `sub` / `job_workflow_ref` pinning, which is the actual control
EKS	`kind`	admission behaviour identical
Secrets Manager + CSI driver	`kind` + the CSI driver's `vault` / `local` provider	delivery path shape is the same; custody model is not
CloudTrail / GuardDuty	none	no equivalent; the Terraform is authored and policy-tested, not applied

production	local / free equivalent	what changes
`terraform validate` / `plan`	`conftest --parser hcl2`	conftest parses the HCL directly, so the IaC gate needs no provider registry and no credentials — which is also why it is the *blocking* IaC gate in CI

Tool substitutions, and why

brief suggested	used	one line
Trivy / Grype / Syft	**Trivy** (SCA + image + IaC), **Syft** (SBOM)	one scanner across three surfaces means one ignore-file semantic for an engineer to learn; Grype dropped as duplicate coverage
Checkov / tfsec	**conftest/OPA** blocking, **Checkov** advisory	conftest runs offline against HCL with no registry or credentials, so the merge decision never depends on network reachability. One tool owns the merge decision; the other adds breadth.
`terraform validate`	**conftest `--parser hcl2`** (+ OpenTofu for `fmt`)	the provider registry is unreachable in a sandbox, so `terraform init` — and therefore `validate` — cannot run offline. Parsing the HCL directly gives a reviewer the identical result with no network.
Kyverno / Gatekeeper	**Kyverno**	`kyverno test` gives real, committed, runnable assertions; Rego for the Terraform side where conftest is the better fit
Falco	**Falco**, custom rules only	the default ruleset describes generic Linux badness; on a twelve-team cluster that is where alert fatigue comes from